

COMP2113 Cheatsheet - Page 1: Linux Commands / Shell / Regex / Makefile / C++ Basics

LINUX COMMAND REFERENCE

```
1. Navigation / help
pwd                                # print absolute current dir
out: /home/user/lab
cd dir | cd .. | cd ~ | cd / # change dir; quote spaces: cd "Module 1"
out: no output if OK; error if dir missing
ls [path]                          # list names in current/path dir
ls -a                              # include hidden .files
ls -lh                             # long format + human sizes
out: -rw-r--r-- 1 u staff 1.2K May 20 a.cpp
man ls                             # manual; /word search, n next, q quit

2. Create / view / edit files
mkdir dir | mkdir a b              # create one/many dirs; no output if OK
mkdir dir                          # remove empty dir only
touch f                             # create empty file/update timestamp
cat f | cat a b                    # print/concatenate file contents
out: hello
less f                              # paged view; space next page, /pat
search, q quit
vi f | vim f | pico f              # terminal editor screen
head -[n N] f                      # first N Lines, default 10; out:
Line1...
tail -[n N] f | tail -f log        # Last N Lines; -f follows appended
lines

3. Copy / move / remove / permission
cp src dst | cp f dir/             # copy file; no output if OK
cp -r dir backup                   # recursive copy directory
mv old new | mv f dir/             # rename or move into existing dir
rm f | rm -r dir | rm -rf d         # remove; -r recursive; -f no
prompt/error
chmod u+x run.sh                  # add owner execute
chmod 755 f                        # owner rwx, group r-x, other r-x

4. Text processing
wc f | wc -l/-w/-c/-m f            # Lines/words/bytes/chars
out: 3 8 42 f
sort f | sort -n | sort -r          # Lexicographic/numeric/reverse sort
out: -t, -k2 data.csv              # comma delimiter, sort by field 2
out: sorted lines
cut -d, -f1,3 data.csv             # print selected fields
out: Alice,95
sort f | uniq | uniq -c             # adjacent duplicate removal/count
out: 3 apple
spell f                            # misspell words; out: recieve
diff expected.txt output.txt       # < first file, > second; out: 2c2

5. Search / print / archive
find . -name "*.cpp" -type f       # recursively print matching file paths
out: ./src/a.cpp
grep -n "pat" f                    # matching lines with line no.; out:
7:hello
grep -E "a|b" f                    # extended regex; grep -i ignores case
echo "$x" | echo -n text           # print text; -n suppresses final
newline
read x | read a b                  # waits for stdin, assigns words to
variables
which g++ | which -a python        # executable path(s); out: /usr/bin/g++
date                               # current date/time; out: Mon May 20
tar -czvf a.tgz *.cpp *.h          # create gzip archive; out lists added
files

Options are command-specific. For example, -d means delimiter in cut, directory test in
[ -d file ], and delete in diff output notation.
```

OPTION MEANING QUICK MAP

```
-a ls -a all incl. hidden; which -a all paths.
-l ls -l long listing; wc -l line count.
-h ls -lh human-readable sizes.
-r rm /cp -r recursive; sort -r reverse.
-f rm -f force; tail -f follow; cut -f field.
-n head/tail -n line count; sort -n numeric; grep -n line no.; echo -n
no newline.
-c wc -c bytes; grep -c count matching lines; g++ -c compile object only.
-w wc -w word count; [ -w f ] writable test.
-m wc -m character count.
-k sort -k2 sort by field 2.
-t sort -t', ' field delimiter for sort.
-d cut -d', ' delimiter; [ -d path ] directory; find -type d dir.
-E grep -E extended regex: +, ?, |, {}.
-P grep -P Perl-compatible regex.
-o grep -o only the matched substring; g++ -o main output name.
-e [ -e f ] exists.
-s [ -s f ] exists and non-empty.
-x [ -x f ] executable; chmod u+x add user execute.
-la ls -la long listing including hidden files.
-rf rm -rf dir recursive + force; dangerous if path is wrong.
- tar: create, gzip, verbose, output filename.
czvf
```

PATHS, PERMISSIONS, FILES

```
absolute path: /home/user/a.cpp
relative path: ../data/input.txt
hidden file: .bashrc
quote spaces: cd "Module 1" or cd Module\ 1
wildcards: * any string, ? one character
cd . stays here; cd .. parent; cd ~ home

-rwxr--r-- 1 jacob sudoer 40 Jan 1 00:00 file.cpp
type: - file, d dir
permission triplets: owner/group/other = rwx/r--/r--
chmod [u/g/o/a][+/-=][r/w/x] file
chmod 755 run.sh # owner rwx, group r-x, other r-x
chmod o+w f      # others add write
chmod +rx f      # all add read+execute
chmod go-rw f    # group/others remove read+write
cp file dir/     # copy into dir
cp -r dir backup # recursive copy
mv old new       # rename if destination absent
rm -rf d         # recursive force delete
vi/vim/pico f    # edit f in terminal
```

REDIRECTION AND PIPES

```
stdin=0 stdout=1 stderr=2
cmd < in.txt      # stdin from file
cmd > out.txt     # stdout overwrite
cmd >> out.txt    # stdout append
cmd 2> err.txt   # stderr overwrite
cmd > out 2> err # split stdout/stderr
cmd > all 2>all  # merge stderr into stdout destination
cmd 1>dev/null 2>all # hide output and error
cmd1 | cmd2      # stdout of cmd1 to stdin of cmd2
cmd1 && cmd2      # run cmd2 only if cmd1 succeeds
cmd1 || cmd2      # run cmd2 only if cmd1 fails
```

PIPELINE PATTERNS

```
sort voting.txt | uniq -c | sort -nr | head -2
grep ' 404 access.log | cut -d ' ' -f1 | sort | uniq -c
sort -k3 -n data.txt | cut -d ' ' -f2,3 > result.txt
ls -l | grep -E '^\.x.x.x'
find . -name '*.cpp' -type f | sort
cut -d' ' -f4-8 mark.txt | ./mark > result.txt
./main < input.txt > output.txt
diff expected.txt output.txt
```

GREP / REGULAR EXPRESSION

```
grep 'hell' f          # basic pattern, case-sensitive
grep -E '[Aa]pple' f   # extended regex
grep -E '^apples' f     # exact whole line
grep -E 'a.*ge' f       # a then any chars then ge
grep -E '(co)+ ' f      # group co repeated 1+ times
grep -E '[0-9]{1,2}[A-Za-z]{3,}[0-9]{4}' f
find . -name 'hello.*' -type f
grep -E '^.{4}$' f      # exactly 4 characters

^ begin line      $ end line      . any char
? zero/one        * zero/more     + one/more
[abc] set         [^abc] not set   [a-z] range
(p){n} exactly   (p){n,} at least n (p){n,m}
a|cd either      . literal dot  [a-z]* lower-case string
\w \d \s word/digit/space
```

COMPACT REGEX APPENDIX

```
^ begin line      $ end line
. any character   \ escape special char
? 0 or 1          * 0 or more      + 1 or more
[abc] set         [^abc] not set   [a-zA-Z] range
(p){n} exactly n  (p){n,} n to infinity
(p){n,m} n to m
a|cd either
\w \W word/nonword
\d \D digit/nondigit
\s \S space/nospace
\n newline
```

SHELL SCRIPT SYNTAX

```
#!/bin/bash
VAR_NAME=value          # no spaces around =
read VAR_NAME           # read stdin
echo "$VAR_NAME"         # double quotes expand variables
echo 'literal $VAR_NAME' # single quotes do not expand
cmd_out="$cmd"           # command substitution
path=Module 1/a.cpp      # double quote paths with spaces
echo "$USER $HOME"       # expands variables, keeps spaces as one arg
echo "$USER $HOME"       # prints literal $USER $HOME
echo \$HOME              # backslash escapes one special char
echo *.cpp               # no wildcard expansion inside quotes
echo *.cpp               # wildcard expands to matching filenames
cmd `date`               # old command substitution; prefer $(date)
${#a}                    # string length
${a:pos:len}             # substring, index starts at 0
${a/find/replace}        # replace first occurrence
${a//find/replace}       # replace all occurrences
let x = x + 1            # arithmetic
x=$((x + 1))             # arithmetic expansion
echo "error" >&2          # output to stderr
exit 1                   # non-zero status means failure
```

ARGUMENTS, TESTS, CONTROL

```
1. Script arguments
$0 script name; $1 first arg; ${10} tenth arg
relative path: ../data/input.txt
$# arg count; $? last exit status; "$@" all args safely
Example: ./run.sh in.txt out.txt
$0= ./run.sh; $1=in.txt; $2=out.txt; $#=2
for a in "$@"; do echo "$a"; done # preserves spaces in args

2. String tests
[ "$s" ]                # non-empty string
[ "$a" == "$b" ]        # string equal
[ "$a" != "$b" ]        # string not equal
[ "$a" = "$b" ]          # lexicographic-order; escape >
Example: if [ "$name" == "Tom" ]; then echo match; fi

3. File and integer tests
[ -e f ] exists; [ -f f ] regular file; [ -d p ] directory
[ -e f ] non-empty; [ -r f ] readable; [ -w f ] writable; [ -x f ]
executable
[ "$s" -eq "$b" ] ; -ne not equal; -lt/-le less; -gt/-ge greater
Example: if [ -f "$1" ] && [ -r "$1" ]; then cat "$1"; fi

4. if / elif / else
if [ cond 1 ]; then cmd; elif [ cond2 ]; then cmd2; else cmd3; fi
Example: if [ "$#" -lt 2 ]; then echo "usage" >&2; exit 1; fi

5. Loops
for i in $list; do cmd; done # word-splitting happens
for i in $(cat); do cmd; done # safer for arguments
for f in *.cpp; do g++ -std=c++11 "$f" -o "${f%.cpp}"; done
for ((i=0; i<n; i++)); do echo "$i"; done
for ((i=10; i<=0; i--)); do sum=$((sum+i)); done
```

COMPILE AND MAKEFILE

```
1. Manual compile commands
g++ -std=c++11 -pedantic-errors -Wall main.cpp -o main
# compile one program; -o names executable
g++ -c gradebook.cpp -o gradebook.o
# compile only to object file, no linking yet
g++ main.o gradebook.o -o main
# link object files into executable
./main < in.txt > out.txt
# run with redirected input/output

2. Typical Makefile
CXX = g++
CXXFLAGS = -std=c++11 -pedantic-errors -Wall
OBJ5 = main.o gradebook.o

main: $(OBJ5)
$(CXX) $(CXXFLAGS) $(OBJ5) -o @

%.o: %.cpp
$(CXX) $(CXXFLAGS) -c %< -o %

main.o: main.cpp gradebook.h
gradebook.o: gradebook.cpp gradebook.h

.PHONY: clean tar
clean:
rm -f *.o main

tar:
tar -czvf submit.tgz *.cpp *.h Makefile

3. Makefile syntax
target: dependencies # if dependency newer, rebuild target
recipe command # recipe line must start with TAB
$@ target name; $< first dependency; $* all dependencies

make # builds first target
make clean # runs clean target
make tar # creates submission archive

4. .h files
Header .h: declarations, struct/class definitions, function prototypes.
Source .cpp: function definitions/implementation.
Use #include "gradebook.h" in .cpp that needs declarations.
Do not compile .h alone; compile .cpp that includes it.
If gradebook.h changes, every .o depending on it must rebuild.
Header guard prevents double include:
#ifndef GRADEBOOK_H
#define GRADEBOOK_H
...
#endif
```

C++ BASICS, ARRAYS, STRINGS

```
#include <cstdlib> # rand, srand
#include <ctime> # time
srand(std::time(nullptr));
rand() % (b - a + 1) + a; # [a,b]

#include <cmath>
sqrt(x); pow(x,y); fabs(x); ceil(x); floor(x);
fmod(x,y); sin(x); cos(x); tan(x); log(x); log10(x);

T arr[n]; T arr[] = {1,2,3}; T arr[3][2];
ret_type f(T arr[], int size);
ret_type g(T arr[][2], int size);

#include <ctype>
isdigit(c); isalpha(c); isalnum(c); islower(c); isupper(c);
isspace(c); isprint(c); tolower(c); toupper(c);

string s = "HKU ENGG1340 CompSc";
s.length()/s.size(); s.empty(); s[i]; s.at(i);
s.substr(pos,len); # substring, len may be omitted
s.find(str,pos=0); s.rfind(str); # return index or string::npos
s.insert(pos,str); s.erase(pos,len); s.replace(pos,len,str);
s = "x"; s.compare(t); # compare: <0, 0, >0
getline(cin,s); # whole line, including spaces
to_string(123); # number -> string: "123"
s.C_str(); # string -> const char* C-string view
string t(ctr); # C-string -> string copy

#include <cstdlib>
abs(x); atoi(ctr); atof(ctr); atol(ctr);
time(nullptr); time(&t); clock();
```

FILE I/O AND FORMATTING

```
1. Open file
#include <fstream>
ifstream fin("input.txt"); # read
ofstream fout("output.txt"); # write, overwrite
ofstream app("log.txt", ios::app); # append
fstream fs("data.txt", ios::in | ios::out);
fin.open(path, C_str_c_str()); # C++11 accepts string too

2. Check open state
if (!fin) { /* open failed */ }
if (fin.fail()) { /* previous operation failed */ }
fin.clear(); # reset state and rewind

3. Read safely
while (fin >> x) { process(x); } # taken / number loop
while (getline(fin, line)) { ... } # whole-line loop
Do not use while(!fin.eof()) as loop condition.

4. Write / redirect
fout << name << ' ' << score << endl; # endl also flushes
cerr << "error\n"; # diagnostic stream

5. Close / flush
fout.flush(); fin.close(); fout.close();
File closes automatically when stream object is destroyed.

6. Parse strings
#include <sstream>
istringstream iss("1 2 3"); iss >> a >> b >> c;
ostringstream oss; oss << fixed << setprecision(2) << x;

7. Formatting
#include <omanip>
cout << 3.1415926; # 3.14159, 6 significant figs
cout << fixed << setprecision(2) << 3.1; # 3.10
cout << scientific << setprecision(2) << 1234.0; # 1.23e+03
cout << showpoint << 3.0; # 3.00000
cout << noshowpoint << 3.0; # 3
cout.unsetf(ios_base::floatfield); # back to default float
cout << setw(5) << 42; # " 42", next item only
cout << setfill('*') << setw(5) << 42; # "****42", fill persists
cout << left << setw(5) << 42; # "42***" if fill is '*'
cout << boolalpha << true; # true; noboolalpha gives 1
```

COMP2113 Cheatsheet - Page 2: Pointers / Linked Lists / STL Containers / C Language

POINTERS AND DYNAMIC MEMORY

```
int val = 5;
int *p = &val; // # p points to val
int *a, b; // # a is pointer; b is int
cout << *p; // # value
cout << p; // # address of val
cout << &p; // # address of pointer
p = nullptr; // # points to nothing
if (p != nullptr) cout << *p; // # check before dereference

int *q = new int; *q = 5; delete q; q = nullptr;
int *arr = new int[n]; delete[] arr; arr = nullptr;
// # increments pointer
(*p)++; // # increments pointed value
a[i] == *(a+i); &a[i] == a+i;
new pairs with delete; new[] pairs with delete[].
```

C++ CORE SYNTAX

Compile: g++ -pedantic-errors -std=c++11 a.cpp -o a
main() starts execution; return 0 means success.
Identifier starts with letter/_; then letter/digit/_; case-sensitive.
Types: char, bool, int, double. Uninitialized values are unknown.
cout << x << endl; cin >> x; // >> stops at whitespace

```
if (cond) stmt; else stmt2;
switch (x) { case v: ...; break; default: ...; }
while (cond) { ... } for (init; cond; update) { ... }
break exits loop/switch; continue jumps to next iteration.
Operators: + / % < > <= >= != && || ! = == --.
Conditional expression: ans = (a > b) ? a : b;
```

REFERENCE VS POINTER SWAP

```
void swapByRef(int& a, int& b) {
    int t = a; a = b; b = t;
}

void swapByPtr(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
```

Call pointer version: swapByPtr(&x, &y);
Reference is an alias and cannot be reassigned.
Pointer stores an address, may be **nullptr**, can point elsewhere.

POINTER PATTERNS

Array as pointer:
int a[10]; int *p = a; p[i] == *(p+i);
Passing array: void fill(int a[], int n); // # array modified in caller

Struct pointer:
struct Student { char name[100]; int uid; };
Student s; Student *sp = &s;
(*sp).uid = 1; sp->uid = 1; // # arrow is shorthand
strcpy(sp->name, "Tim"); // # cannot assign char array by =

C dynamic memory:
int *p = malloc(n * sizeof *p);
if ((p) exit(1));
free(p); p = nullptr;
malloc/calloc/realloc return void*; C auto-converts, C++ needs cast/new.

POINTER PITFALLS

Dangling pointer: pointer still stores freed/deleted address.
Memory leak: allocated memory is no longer reachable.
Double free/delete: releasing same allocation twice is undefined.
Wrong deallocation pair: new[] must use delete[], malloc must use free.
Off-by-one: last valid index is n-1; a+n points one past last only.
Never dereference one-past-last, **nullptr**, or an **uninitialized** pointer.

Dynamic array resize pattern:
T *b = new T[newSize];
copy old items; delete[] a; a = b; b = nullptr;

LINKED LIST POINTER RULES

Use cur for traversal; do not move head unless deleting/changing head.
while (cur != **nullptr**) { process(cur); cur = cur->next; }
For deletion keep prev:
if (prev == **nullptr**) head = cur->next; else prev->next = cur->next;
delete old node after saving next pointer.

head insert: n->next = head; head = n;
tail insert: if empty head=tail=n; else tail->next=n; tail=n;
Sequential access: kth node needs walking from head, O(k).
Array random access O(1); linked-list insert/delete after node O(1).

FUNCTIONS, SCOPE, PASSING

```
ret_type f(type1 p1, type2 p2) { statements; return value; }
void f(...) { statements; return; } // return optional in void
Function call is an expression: z = larger(2.5, larger(a,b));
Prototype before use: double larger(double, double);
```

Local vars: inside block/function; disappear after block exits.
Global vars: outside functions; visible after declaration, avoid for data flow.
Inner block variable hides same-name outer variable.

Pass by value: void f(int x) // copy, caller unchanged
Pass by ref: void f(int& x) // C++ alias, caller changed
Pass pointer: void f(int *p) // C/C++, call f(&x), use *p

LINEAR SEARCH / SELECTION SORT

```
int linSearch(T a[], int n, T x) {
    for (int i = 0; i < n; ++i)
        if (a[i] == x) return i;
    return -1;
}

void selSortAsc(T a[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int m = i;
        for (int j = i + 1; j < n; ++j)
            if (a[j] < a[m]) m = j;
        if (m != i) { T t = a[i]; a[i] = a[m]; a[m] = t; }
    }
}
```

LINKED LIST BASICS

```
struct Node { int data; Node* next; };
Node* head = nullptr;
Node* n = new Node;
n->data = 10; n->next = nullptr; head = n;

Node* cur = head;
while (cur != nullptr) {
    doSomething();
    cur = cur->next;
}

void insertAtFront(Node& head, Node* n) {
    n->next = head;
    head = n;
}
```

LINKED LIST INSERT / DELETE

```
void insertAtEnd(Node& head, Node* n) {
    Node* cur = head;
    while (cur != nullptr) {
        if (cur->data == val) {
            if (prev == nullptr) head = cur->next;
            else prev->next = cur->next;
            delete cur;
            cur = prev; prev->next = head;
            continue; // delete all occurrences
        }
        prev = cur; cur = cur->next;
    }
}
```

LINKED LIST CLEAR / REVERSE

```
void deleteList(Node& head) {
    Node *cur = head, *next;
    while (cur != nullptr) {
        next = cur->next;
        delete cur;
        cur = next;
    }
    head = nullptr;
}

void reverseList(Node& head) {
    Node *cur = head, *prev = nullptr, *next;
    while (cur != nullptr) {
        next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    head = prev;
}
```

STRUCT IN C++

```
struct Point {
    int x, y, z;
    Point(int x, int y, int z) : x(x), y(y), z(z) {}
    double distance(const Point& p) {
        return sqrt(pow(x-p.x,2)+pow(y-p.y,2)+pow(z-p.z,2));
    }
    bool operator<(const Point& p) {
        return x < p.x || (x == p.x && y < p.y) ||
            (x == p.x && y == p.y && z < p.z);
    }
};

Point p1(1,2,3), p2(4,5,6);
double dist = p1.distance(p2);
```

STL ITERATORS

STL = containers + iterators + algorithms.
Iterator belongs to one container type; do not mix vector/list/map iterators.

vector<int>::iterator it = v.begin();
cout << *it; *it = 100; ++it; // # dereference changes item
c.begin() points first item; c.end() points one past last, not a real item.
Never dereference end(); use it != c.end() as loop condition.

Forward: =, ++, *, *, =, !=
Bidirectional: forward + -, --, <, <=, >, >=, ==, !=
Random access: bidirectional + +, -, --, <, <=, >, >=, ==, !=
For vector, v.end()-v.begin() == v.size(); for list/map/set this is invalid.
Iterator is not necessarily a memory address; it hides container structure.

RECURSION PATTERNS

Recursive function = base case + recursive call on smaller input.
Must eventually reach base case; otherwise stack **overflow**.

```
int fact(int n) { if (n == 0) return 1; return n * fact(n-1); }
int fib(int n) { if (n < 2) return n; return fib(n-1)+fib(n-2); }
int gcd(int x, int y) { if (y == 0) return x; return gcd(y,x%y); }
bool pal(string s) {
    return s.length() < 2 ||
           (s[0] == s[s.length()-1] && pal(s.substr(1,s.length()-2)));
}
```

VECTOR / LIST / SET / MAP

```
#include <vector>
vector<int> v;
v.push_back(x); v.pop_back(); v[i]; v.at(i); v.size(); v.empty();
v.front(); v.back(); v.capacity(); v.resize(n); v.reserve(n);
v.insert(pos,val); v.erase(pos); v.clear(); v.swap(v2);
push_back/pop_back/index/size are O(1); push_back copies the item.
Pros: random access O(1), compact memory, works with std::sort.
Cons: middle insert/erase costly; reallocation may invalidate iterators.
Use vector when random access and sorting are needed.
for (vector<T>::iterator it = v.begin(); it != v.end(); ++it)
    cout << *it;
for (auto it = v.begin(); it != v.end(); )
    if (condition) it = v.erase(it); else ++it;
```

```
#include <list>
list<int> l;
l.push_back(1); l.push_front(2);
l.pop_back(); l.pop_front(); l.size(); l.front(); l.back();
l.sort(); l.reverse(); // # list member sort/reverse
list<T>::iterator it; // # bidirectional, no l[i], no random access
Pros: O(1) insert/erase at known position; stable iterators.
Cons: no index/random access; extra pointer memory; cannot std::sort.
Use list for frequent insertion/deletion, not kth-item access.
```

```
#include <set>
set<int> s;
s.insert(3); s.insert(3); // duplicate ignored
s.count(3); s.find(3); s.erase(3); s.size(); s.empty();
for (auto it = s.begin(); it != s.end(); ++it) cout << *it;
Set stores unique keys in sorted order; key type needs operator<.
Default order is ascending; rbegin/rend visits reverse order.
Pros: unique + sorted + O(log n) find/insert/erase.
Cons: no duplicates, no index access, cannot modify key through iterator.
```

```
#include <map>
map<string,int> m;
m["Alan"] = 90; // insert/update value by key
m.count("Alan"); m.find("Alan"); m.erase("Alan"); m.size();
for (auto it = m.begin(); it != m.end(); ++it)
    cout << it->first << it->second; // ascending key order
m[key] creates default value if key absent; use count/find to test first.
map keeps keys sorted; begin() gives smallest key.
Pros: key-value lookup/update O(log n), sorted by key.
Cons: no duplicate keys; m[key] inserts default if absent.
```

STL ALGORITHMS / SORT DETAILS

```
#include <algorithm>
sort(v.begin(), v.end()); // default ascending by operator<
sort(v.begin(), v.end(), greater<int>()); // descending
sort(a, a+n); // array pointers act as iterators
sort(v.rbegin(), v.rend()); // descending via reverse iterators
sort(v.begin(), v.end(), greater<int>()); // descending comparator
reverse(v.begin(), v.end()); // reverse current order, no sorting
std::find(v.begin(), v.end(), x); // linear search, returns iterator/endl
```

Do not call std::sort on list/map/set iterators; they are not random access.
Use l.sort() for list sorting; l.reverse() reverses list order.
map/set are already sorted by key; iteration visits ascending keys.
Use rbegin()/rend() to visit vector/list/set/map in reverse order.
For user-defined keys/items used in set/map/sort, define operator<:
bool operator<(const Email& a, const Email& b) {
 if (a.sender != b.sender) return a.sender < b.sender;
 return a.subject < b.subject;
}

Complexity: vector random access O(1); list kth access O(k);
map/set insert/find/erase O(log n); sort average O(n log n).

```
int binSearch(int a[], int l, int r, int x) {
    if (r >= l) {
        int m = l + (r-l)/2;
        if (a[m] == x) return m;
        if (a[m] > x) return binSearch(a, l, m-1, x);
        return binSearch(a, m+1, r, x);
    }
    return -1;
}
```

C LANGUAGE

```
#include <stdio.h>
int a; char s[100], line[100];
scanf("%d", &a); scanf("%99s", s); // width prevents overflow
printf("a = %d\n", a);
fgetc(line, sizeof line, stdin); // reads spaces, nullopt if fail
line[strlen(line),"n"] = '\0'; // remove trailing newline
```

```
%d int %f/%lf float/double
%.nf n decimals %g no trailing 0
%c char %n(n)s width n string
sprintf(buf, "%d", x); sscanf(str, "%d", &x);
remove("a.txt"); rename("old", "new");
```

```
#include <string.h>
strcpy(s1,s2); strcat(s1,s2); strcmp(s1,s2); strlen(s);
strchr(s,ch); strchr(s,ch); strstr(s,sub); strtok(s,delim);
strncpy/strncat need manual '\0' check if buffer may fill.
```

```
#include <stdlib.h>
void *malloc(size_t size); // uninitialized bytes
void *calloc(size_t n, size_t size); // zero-initialized n items
void *realloc(void *p, size_t newsize); // may move allocation
void free(void *p); // freeing empty pointer is safe
int *p = malloc(n * sizeof *p); // C: no cast needed
if (!p) exit(1);
free(p); p = nullptr; // avoid dangling pointer
```

```
int *q = calloc(n, sizeof *q); // all elements start as 0
int *tmp = realloc(p, newN * sizeof *p);
if (tmp != nullptr) p = tmp; // keep old p if realloc fails
realloc(nullptr, size) == malloc(size); realloc(p,0) is free-like.
```

```
struct point { int x, y; };
typedef struct point Point;
# C structs cannot have member functions/constructors.
#include <limits.h> INT_MAX INT_MIN CHAR_BIT ULONG_MAX
Compile C math library: gcc file.c -o file -lm
```

C VS C++ DIFFERENCES

- Language/library
C has no cin/cout, string, vector, reference &, new/delete, class methods.
Use printf/scanf/fgetc, char arrays, **malloc/free**, struct + functions.
- Functions and arrays
C function must be declared before use; no default pass-by-reference.
Array parameter becomes pointer: void f(int a[]) == void f(int *a).
sizeof(array) works only in same scope; in function it is pointer size.
- Memory and strings
void* from malloc auto-converts in C; C++ requires cast or use new.
char array cannot use = after declaration; use strcpy/strncpy.
- Input traps
scanf("%s", s) no & for char array; scanf("%d", &x) needs address.
Mixing scanf then fgetc leaves newline; consume newline first.
- Safety
Check every allocation and file open before dereference/use.

CHAR ARRAY / C-STRING

```
char name[] = "John"; // auto size incl. '\0'
char name[16] = "John"; // fixed buffer
name[i] accesses one char; name[2] = 'a';
cin >> name; scanf("%15s", name); // no spaces, no & for char array
cout << name; printf("%s", name);
```

C-string ends with '\0'; ensure enough space for terminator.
strlen(s): count chars before '\0', not including terminator.
strcpy(dest, src): copy src incl. '\0'; dest must be large enough.
strncpy(dest,src,n): may omit '\0' if full; check terminator manually.
strcat(dest, src): append src; dest must have remaining capacity.
strcpy(a,b): @ if a=b, @ equal, @ if a=b; do not use ==.
strchr(s,c): first char c; strchr(s,c): last char c.
strstr(s,sub): first substring; returns pointer to **nullptr**.
strtok(s,delim): tokenizes by writing '\0' into s; destroys original.

Must use / need C-string when:

- C library API: printf/scanf/fgetc/strlen/strcpy/strncpy.
 - char buffer required: char a[100], fixed memory, manual '\0'.
 - argv and legacy APIs; older file open may need path.c_str().
- Conversions:
string s = "abc"; const char *p = s.c_str(); // valid while s unchanged
char buf[20]; strcpy(buf, s.c_str()); // string -> writable char[]
string t(buf); int x = stoi(t); string u = to_string(123);
Manual upper to lower:
if ('A' <= c && c <= 'Z') c = c + ('a' - 'A');

EXAM PITFALLS

- Input
cin >> s stops at whitespace; getline(cin,s) reads line.
scanf("%s",s) may **overflow** unless **width** is given: "%99s".
After scanf, fgetc may read leftover '\n'; consume or handle newline.
- Pointers / memory
int *a, b; means only a is pointer; write int *a, *b for two pointers.
Never dereference **nullptr/uninitialized/one-past-end** pointer.
new[] pairs with delete[]; **malloc/calloc/realloc** pairs with **free**.
Do not assign **realloc** result directly to p before checking **nullptr**.
- Arrays / strings
sizeof(array) inside function is pointer size, not element count.
char array cannot be compared/assigned by == or !=; use strcmp/strcpy.
Last valid array index is **size-1**; loop condition usually i < size.
- STL / algorithms
m[key] creates default value; use count() to test existence.
sort uses half-open range [first,last); last iterator not included.
- Testing / output
Test empty, one item, duplicates, max size, no match, all same.
Use diff to catch invisible spaces, missing newline, and line breaks.
Use long long for large counts/totals; watch int division truncation.